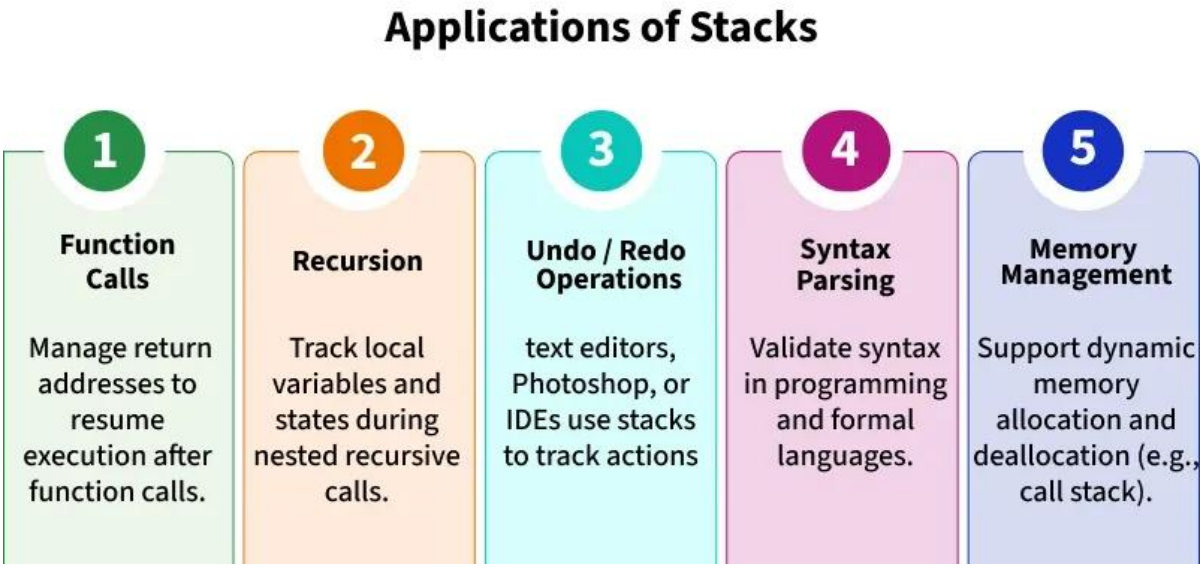
Stack: A Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO(Last In First Out) or FILO(First In Last Out). LIFO implies that the element that is inserted last, comes out first and FILO implies that the element that is inserted first, comes out last.

Types of Stack:
1.Fixed Size Stack: A fixed size stack has a predefined capacity. Once it becomes full, no more elements can be added (this causes **overflow**). If the stack is empty and we try to remove an element, it causes **underflow**. Typically implemented using a static array.
2.Dynamic Size Stack: A dynamic size stack can grow and shrink automatically as needed. If the stack is full, its capacity expands to allow more elements. As elements are removed, memory usage can shrink as well. Can be implemented using:
Linked List: grows/shrinks naturally.
Dynamic Array (like vector in C++ or ArrayList in Java): resizes automatically.

Stack primitive functions:
1.push(item): Inserts (adds) an element into the stack.
2.pop(): Removes (deletes) the top element from the stack.
3.peek() / top(): Returns the top element of the stack without removing it.
4.isEmpty(): Checks if the stack is empty. Returns true if stack has no elements, otherwise false.
5.isFull() (only for stacks with fixed size, e.g., array implementation): Checks if the stack is full. Useful in static stack implementations.

Stack Applications:



Infix Expression: Infix expressions are mathematical expressions where the operator is placed between its operands. This is the most common mathematical notation used by humans. For example, the expression "2 + 3" is an infix expression, where the operator "+" is placed between the operands "2" and "3".

Prefix Expression: Postfix expressions are also known as Reverse Polish Notation (RPN), are a mathematical notation where the operator follows its operands. In postfix notation, operands are written first, followed by the operator. For example, the infix expression "5 + 2" would be written as "5 2 +" in postfix notation.

Postfix Expression: Prefix expressions are also known as Polish notation, are a mathematical notation where the operator precedes its operands. In prefix notation, the operator is written first, followed by its operands. For example, the infix expression "a + b" would be written as "+ a b" in prefix notation.

Conversion of Infix to Postfix Expression:

1.Scan the infix expression from left to right.

2.If the scanned character is an operand, put it in the postfix expression.

3.Otherwise, do the following:

- If the precedence of the current scanned operator is higher than the precedence of the operator on top of the stack, or if the stack is empty, or if the stack contains a '(', then push the current operator onto the stack.
- Else, pop all operators from the stack that have precedence higher than or equal to that of the current operator. After that push the current operator onto the stack.

4.If the scanned character is a '(', push it to the stack.

5.If the scanned character is a ')', pop the stack and output it until a '('is encountered, and discard both the parenthesis.

6.Repeat steps 2-5 until the infix expression is scanned.

7.Once the scanning is over, Pop the stack and add the operators in the postfix expression until it is not empty.

8.Finally, print the postfix expression.

Conversion of Infix to Prefix Expression:

1.Reverse the infix expression. Note while reversing each '(' will become ')' and each ')' becomes '('.

2.Convert the reversed infix expression to postfix expression, for that used the infix to postfix algorithm.

3.Reverse the postfix expression.

4.The final result is the Prefix expression.

Evaluation of Postfix Expression:

1.Create a stack to store operands (values).

2.Scan the given expression from left to right and do the following for every element in array.

- If the element is a number, push it into the stack.
- If the element is an operator, pop operands for the operator from the stack. Evaluate the operator and push the result back to the stack.

3.When the expression is ended, the number in the stack is the final answer.

Evaluation of Prefix Expression:

1.Create a stack to store operands (values).

2.Scan the given expression from right to left and do the following for every element in array.

- If the element is a number, push it into the stack.
- If the element is an operator, pop operands for the operator from the stack. Evaluate the operator and push the result back to the stack.

3.When the expression is ended, the number in the stack is the final answer.

Queue: A queue is a linear data structure that adheres to the First-In, First-Out (FIFO) principle. This means that the element inserted first into the queue will be the first one to be removed.

Basic Operations:

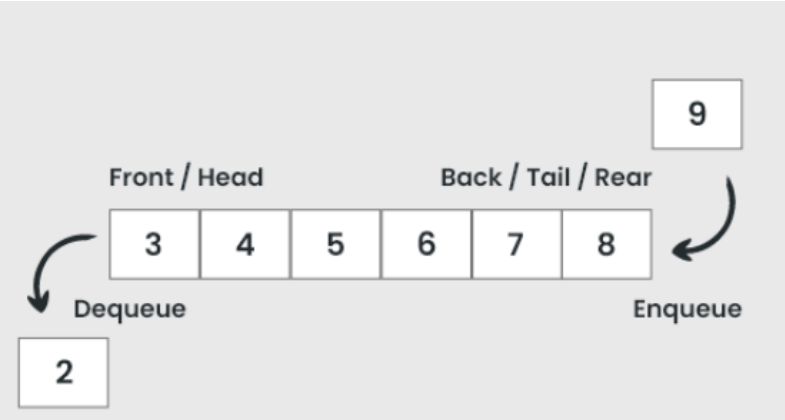
1.Enqueue: Adds an element to the rear of the queue.

2.Dequeue: Removes an element from the front of the queue.

3.Peek (or Front): Returns the element at the front of the queue without removing it.

4.isEmpty: Checks if the queue is empty.

5.isFull: Checks if the queue is full (relevant for array-based implementations).



Types of Queues:

1.Priority Queue: A priority queue is a type of queue that arranges elements based on their priority values. Each element has a priority associated. When we add an item, it is inserted in a position based on its priority. Elements with higher priority are typically retrieved or removed before elements with lower priority.

Types of it:

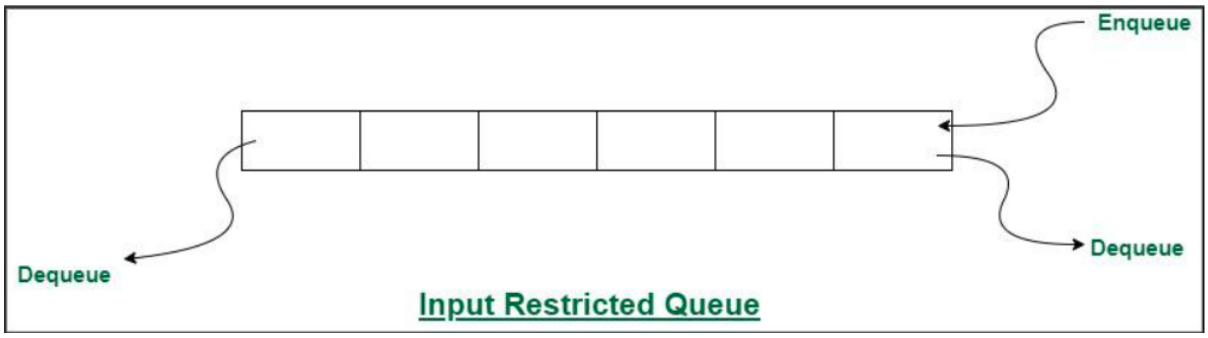
i.Ascending Order Priority Queue: In this queue, elements with lower values have higher priority. For example, with elements 4, 6, 8, 9, and 10, 4 will be dequeued first since it has the smallest value, and the dequeue operation will return 4.

ii.Descending order Priority Queue: Elements with higher values have higher priority. The root of the heap is the highest element, and it is dequeued first. The queue adjusts by maintaining the heap property after each insertion or deletion.

2.Deque (Double-Ended Queue): A Deque (Double-Ended Queue) is a linear data structure that allows elements to be added or removed from both ends—front and rear. Unlike a regular queue that adds and removes elements from one end (FIFO) or a stack that works by adding and removing elements from the top (LIFO), a deque allows you to add or remove elements from both ends, offering more flexibility.

Types of it:

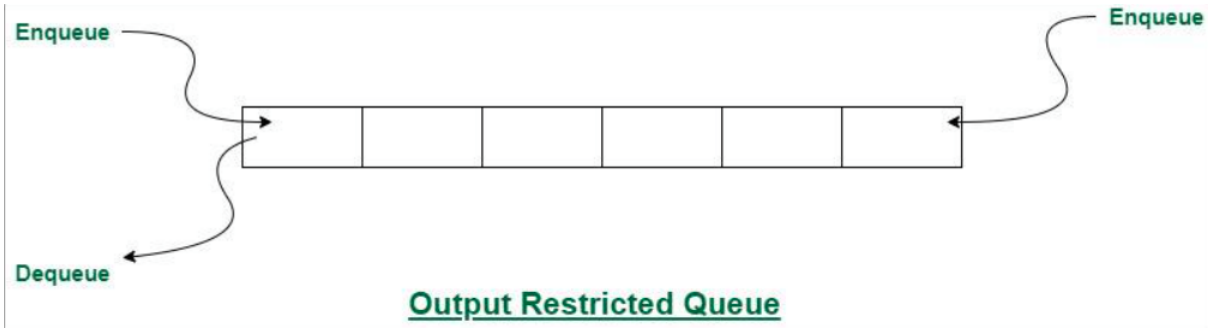
i.Input Restricted Queue: An input restricted queue is a special case of a double-ended queue where data can be inserted from one end(rear) but can be removed from both ends (front and rear). This kind of Queue does not follow FIFO(first in first out).



Mainly the following three basic operations are performed on input restricted queue:

- **insertRear():** Adds an item at the rear of the queue.
- **deleteFront():** Deletes an item from the front of the queue.
- **deleteRear():** Deletes an item from rear of the queue.

ii.Output Restricted Queue: An output-restricted queue is a special case of a double-ended queue where data can be removed from one end(front) but can be inserted from both ends (front and rear). This kind of Queue does not follow FIFO(first in first out).



Mainly the following three basic operations are performed on output restricted queue:

- **insertRear():** Adds an item at the rear of the queue.
- **insertFront():** Adds an item at the front of the queue.
- **deleteFront():** Deletes an item from the front of the queue.

3.Circular Queue: A circular queue is a type of queue in which the last position is connected back to the first position to make a circle. Unlike a simple linear queue where unused spaces cannot be reused once elements are dequeued, a circular queue efficiently utilizes memory by reusing the empty spaces created at the front. It follows the FIFO (First In First Out) principle but with the additional feature that the rear pointer wraps around to the beginning when it reaches the end of the queue.

4.Linear Queue: A linear queue is the simplest form of a queue that works on the FIFO principle, which means First In First Out. In this structure, insertion of elements is done at the rear end, and deletion of elements is done from the front end. It is called linear because the elements are arranged sequentially in a straight line.

NOTE: Main Disadvantage: Linear queue wastes memory space because once elements are deleted from the front, those positions cannot be reused. This is overcome by Circular queue!

Static Implementation of Static Queue:

1.A static queue is implemented using a fixed-size array.
2.Two pointers, front and rear, are maintained to keep track of the queue.
3.Insertion is done at the rear end and deletion at the front end.
4.The size of the queue is fixed, so memory cannot be increased at runtime.
5.Main drawback is wastage of memory because once space is freed from the front, it cannot be reused.

Dynamic Implementation of Queue

1.A dynamic queue is implemented using linked lists instead of arrays.
2.Each element is stored in a node containing the data and a pointer to the next node.
3.Memory is allocated at runtime, so the queue can grow or shrink as needed.
4.No wastage of memory occurs because nodes are created and deleted dynamically.
5.However, it requires extra memory for storing pointers and has more implementation complexity.

Static Implementation of Circular Queue

1.A circular queue is implemented using a fixed-size array but the last position is connected back to the first to form a circle.
2.Two pointers, front and rear, are maintained, and they wrap around when they reach the end of the array.
3.Insertion is performed at the rear and deletion at the front, but unlike linear queue, freed spaces can be reused.
4.This eliminates the problem of memory wastage present in a simple linear queue.
However, the size is still fixed, as it depends on the array size chosen during implementation.

Applications of Queue:

1.Task Scheduling: Queues can be used to schedule tasks based on priority or the order in which they were received.
2.Resource Allocation: Queues can be used to manage and allocate resources, such as printers or CPU processing time.
3.Operating systems: Operating systems often use queues to manage processes and resources. For example, a process scheduler might use a queue to manage the order in which processes are executed.
4.Printer queues: In printing systems, queues are used to manage the order in which print jobs are processed. Jobs are added to the queue as they are submitted, and the printer processes them in the order they were received.
5.Network protocols: Network protocols like TCP and UDP use queues to manage packets that are transmitted over the network. Queues can help to ensure that packets are delivered in the correct order and at the appropriate rate.
6.Web servers: Web servers use queues to manage incoming requests from clients. Requests are added to the queue as they are received, and they are processed by the server in the order they were received.

Applications of Stack:

1.Function calls: Stacks are used to keep track of the return addresses of function calls, allowing the program to return to the correct location after a function has finished executing.

2.Recursion: Stacks are used to store the local variables and return addresses of recursive function calls, allowing the program to keep track of the current state of the recursion.

3.Expression evaluation: Stacks are used to evaluate expressions in postfix notation (Reverse Polish Notation).

4.Syntax parsing: Stacks are used to check the validity of syntax in programming languages and other formal languages.

5.Memory management: Stacks are used to allocate and manage memory in some operating systems and programming languages.